DAA-Practical-Assignment-Report

19 March 2026

Team Members

1. 241CS250, S Ashwin, sashwin.241cs250@nitk.edu.in, 8088437882
2. 241CS252, Samarth Talawar, samarthpt216@gmail.com, 8217215496

Introduction

Sorting is a fundamental operation in computer science and plays a crucial role in optimizing the performance of many algorithms and applications. Efficient sorting enables faster searching, better data organization, and improved computational efficiency in a wide range of systems such as databases, operating systems, and data processing pipelines. Because of its importance, a variety of sorting algorithms have been developed, each with different theoretical complexities and practical performance characteristics.

This report focuses on a comparative study of several widely used sorting algorithms, namely Selection Sort, Bubble Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort, and Radix Sort. While theoretical time complexities provide a general understanding of algorithm efficiency, actual performance can vary significantly depending on factors such as input size, input distribution, and system environment. Therefore, experimental analysis is essential to evaluate how these algorithms behave in real-world scenarios.

The primary objective of this study is to analyze and compare the performance of these sorting algorithms using different types of input data, including randomly generated elements, already sorted elements, and reverse-sorted elements. Special emphasis is placed on Quick Sort, where different pivot selection strategies are implemented and evaluated to understand their impact on performance.

Through systematic experimentation and analysis, this report aims to bridge the gap between theoretical expectations and practical observations, providing insights into the strengths and limitations of each sorting algorithm under various conditions.

Data Generation and Experimental setup

To evaluate and compare the performance of different sorting algorithms, a structured experimental setup was designed. The goal was to ensure consistency, fairness, and reliability in the observed results.

Machine Specifications

All experiments were conducted on a system equipped with an Intel Core Ultra 9 185H processor, featuring a 24 MB cache and a base clock speed of 2.3 GHz. The performance measurements obtained are therefore dependent on this specific hardware configuration.

Timing Mechanism

The execution time of each sorting algorithm was measured using high-resolution timing functions available in C++. Specifically, the chrono library was used to record the start and end times of each sorting operation, allowing precise computation of elapsed time in microseconds.

Repetition of Experiments

To minimize the impact of system noise and variability (such as background processes), each sorting algorithm was executed 10 times for the same input. The average of these runs was computed within the program and used as the final execution time for analysis. This approach improves the reliability and stability of the results.

Reported Times

The execution times of all sorting algorithms were recorded in microseconds for different input sizes - 100, 500, 1000, 5000, 7500, 10000 and 20000, and input orders (increasing, decreasing, and random), as shown in Tables 1 to 6.

A clear trend observed across all tables is that execution time increases with input size for every algorithm. However, the rate of increase varies significantly depending on the algorithm and input distribution.

Tables 1 to 3 showcase reported times of the best quick sort version i.e., with median pivot and the remaining sorting algorithms. Tables 4 to 6 showcase comparision between different versions of quick sort algorithms.

Reported Times for Increasing Input (in micro seconds)							
Sorting Algorithms	100	500	1000	5000	7500	10000	20000
Selection Sort	0	600	1599	32282	67160	122717	483181
Bubble Sort	0	599	1400	32344	70935	130057	525642
Insertion Sort	0	0	0	118	100	0	100
Merge Sort	0	300	299	2000	3051	3353	7237
Quick Sort (Median Pivot)	0	0	100	751	1000	810	1600
Heap Sort	0	100	251	1148	2322	2464	6151
Radix Sort	144	0	199	300	555	1199	1000

Table 1: Sorting Algorithm Performance for Increasing Order

Reported Times for Decreasing Input (in micro seconds)							
Sorting Algorithms	100	500	1000	5000	7500	10000	20000
Selection Sort	0	600	2551	35356	79625	144948	688206
Bubble Sort	99	900	3400	83761	187982	336255	1334560
Insertion Sort	0	600	1799	44107	100696	175269	703671
Merge Sort	0	300	300	2800	2200	3252	6651
Quick Sort (Median Pivot)	0	100	100	799	801	1054	2400
Heap Sort	0	100	200	1001	1965	2298	4699
Radix Sort	0	0	100	200	499	500	1000

Table 2: Sorting Algorithm Performance for Decreasing Order

Reported Times for Random Input (in micro seconds)							
Sorting Algorithms	100	500	1000	5000	7500	10000	20000
Selection Sort	110	499	1500	31927	73196	123832	487224
Bubble Sort	99	1000	2800	80283	182190	337048	1397540
Insertion Sort	0	299	899	22101	48469	86747	342315
Merge Sort	100	100	600	2300	3009	4018	7951
Quick Sort (Median Pivot)	0	0	100	600	900	1400	2753
Heap Sort	0	0	199	1300	1899	2700	5500
Radix Sort	100	0	0	199	399	462	1000

Table 3: Sorting Algorithm Performance for Random Order

Quick Sort Variants (Increasing Input, in micro seconds)							
Variant	100	500	1000	5000	7500	10000	20000
First Pivot	0	238	699	15650	34807	65819	245097
Random Pivot	0	0	100	399	500	700	1851
Median Pivot	0	0	0	300	599	499	900

Table 4: Quick Sort Performance for Increasing Input

Quick Sort Variants (Decreasing Input, in micro seconds)							
Variant	100	500	1000	5000	7500	10000	20000
First Pivot	99	699	1899	43125	101269	158416	571968
Random Pivot	0	0	100	550	499	1300	1600
Median Pivot	0	0	100	200	500	400	1102

Table 5: Quick Sort Performance for Decreasing Input

Quick Sort Variants (Random Input, in micro seconds)							
Variant	100	500	1000	5000	7500	10000	20000
First Pivot	99	0	299	900	799	1050	2399
Random Pivot	0	0	199	699	900	1300	3100
Median Pivot	0	0	200	800	801	1199	2550

Table 6: Quick Sort Performance for Random Input

Input Generation

Test cases were generated programmatically using C++ file handling (fstream). The input files were structured to support multiple test cases within a single file. The format used is as follows:

- The first line contains the total number of test cases, n
- For each test case:
  - A line specifying the size of the array (k)
  - A line containing k space-separated integers

For the experiments, a total of 7 different input sizes were considered: 100, 500, 1000, 5000, 7500, 10000, and 20000 elements. These sizes were chosen to observe the behavior of algorithms across both small and large datasets.

Consistency of Inputs

To ensure fairness in comparison, the same input files were used across all sorting algorithms. This eliminates bias that could arise from differences in input data and ensures that performance differences are solely due to the algorithms themselves.

Analysis of various versions of Quick Sort

Figures 1, 2, and 3 illustrate the performance of Quick Sort using three different pivot selection strategies: first element, random element, and median-of-three, for increasing, decreasing, and random inputs respectively.

From Figure 1 (Increasing Input), the Quick Sort version using the first element as pivot performs very poorly as input size increases. This is expected, since for already sorted data, choosing the first element leads to highly unbalanced partitions and worst-case $O(n^2)$ behavior. In contrast, both random pivot and median-of-three pivot strategies significantly improve performance by producing more balanced partitions. Among these, the median-of-three pivot performs slightly better and shows more stable growth.

In Figure 2 (Decreasing Input), a similar trend is observed. The first pivot strategy again shows worst-case behavior, with execution time increasing sharply as n grows. The random pivot performs better, but still shows some variability. The median-of-three pivot consistently outperforms the other two, maintaining lower execution times and more stable performance across all input sizes.

From Figure 3 (Random Input), all three versions perform closer to their average-case O(nlogn) complexity. The difference between the three strategies is less pronounced compared to the sorted cases. However, the median-of-three pivot still maintains a slight advantage, followed by the random pivot, while the first pivot version remains the least efficient.

Overall, the analysis clearly shows that pivot selection has a significant impact on Quick Sort performance. The median-of-three pivot strategy provides the best and most consistent performance across all input types, while choosing the first element as pivot leads to poor performance in unfavorable input conditions.

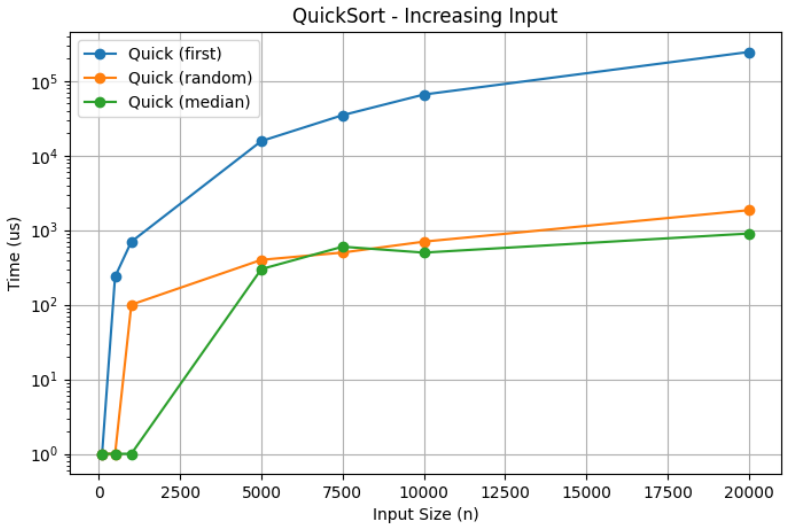

Figure 1: Various Versions of Quick Sort Algorithms Graph with Increasing order of elements

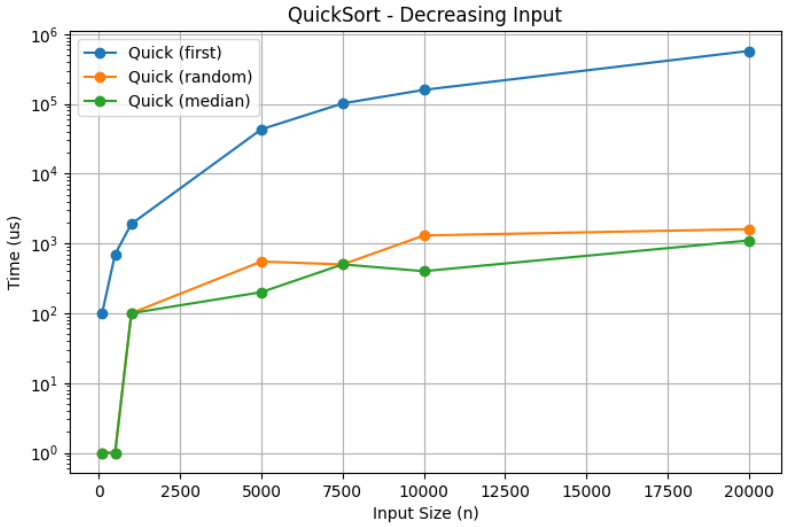

Figure 2: Various Versions of Quick Sort Algorithms Graph with Decreasing order of elements

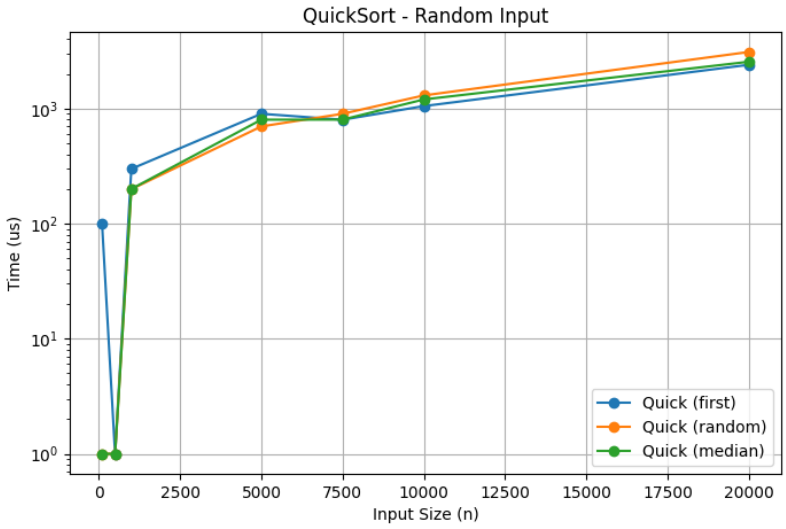

Figure 3: Various Versions of Quick Sort Algorithms Graph with Random order of elements

Analysis of Sorting Algorithms

Figures 4, 5, and 6 illustrate the variation of execution time with respect to input size n for increasing, decreasing, and random input orders respectively.

From Figure 4 (Increasing Input), it is observed that Selection Sort and Bubble Sort exhibit a steep increase in execution time as n grows, confirming their $O(n^2)$ behavior. Insertion Sort performs significantly better in this case due to the input already being sorted, resulting in minimal shifts and near-linear performance. Merge Sort, Heap Sort, and Quick Sort (median pivot) show relatively stable and efficient growth, maintaining O(nlogn) performance. Radix Sort performs the best overall, showing very low execution time even for large inputs.

In Figure 5 (Decreasing Input), the performance of Insertion Sort degrades drastically compared to the increasing case, as it now has to perform the maximum number of shifts. Similarly, Quick Sort with poor pivot choices (if considered) would approach worst-case behavior. However, the median pivot version still maintains stable performance. Merge Sort and Heap Sort remain largely unaffected by input order, demonstrating their consistency. Radix Sort again shows strong performance with minimal growth in execution time.

From Figure 6 (Random Input), which represents the average-case scenario, all algorithms behave closer to their expected theoretical complexities. Quadratic algorithms (Selection, Bubble, Insertion) show rapid growth in execution time, making them impractical for large inputs. Efficient algorithms like Merge Sort,

Heap Sort, and Quick Sort (median pivot) scale much better. Among these, Quick Sort with median pivot performs competitively, while Radix Sort continues to outperform comparison-based algorithms for larger input sizes.

Across all three figures, a clear distinction can be seen between $O(n^2)$ and O(nlogn) algorithms. Additionally, the graphs highlight the sensitivity of certain algorithms (like Insertion Sort and Quick Sort) to input order, whereas others (like Merge Sort and Heap Sort) remain stable regardless of input distribution.

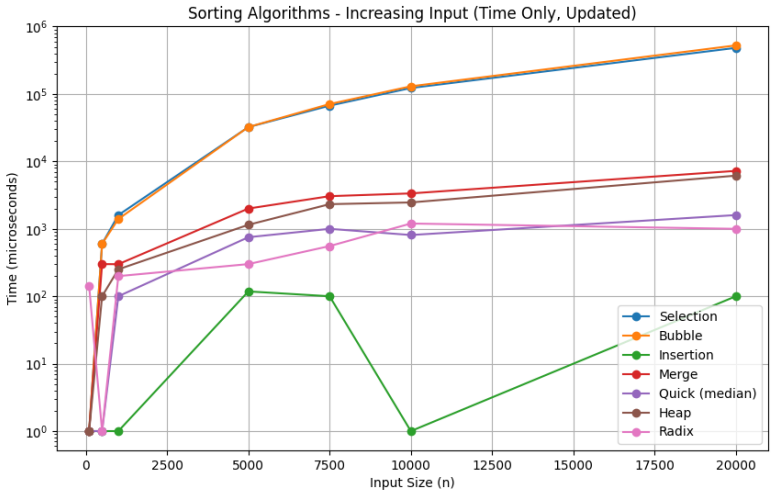

Figure 4: Graph of various Sorting Algorithms with Increasing order of elements as input

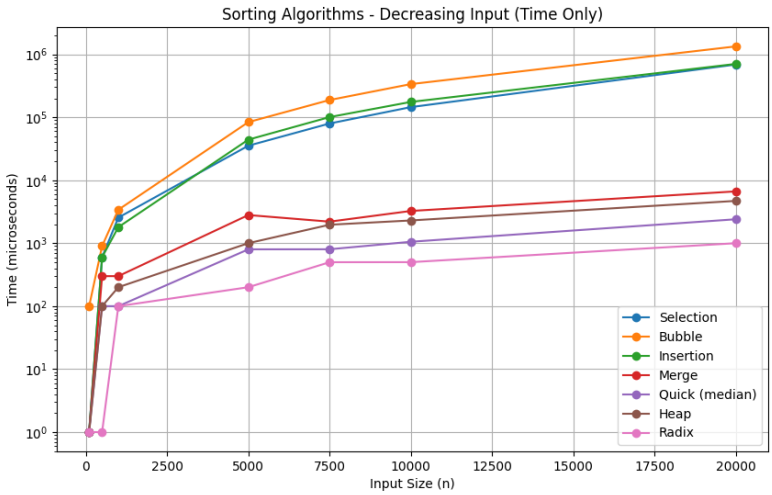

Figure 5: Graph of various Sorting Algorithms with Decreasing order of elements as input

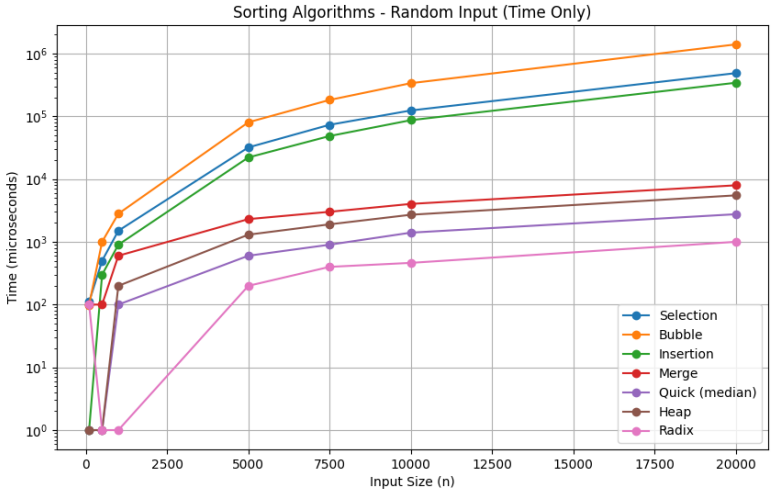

Figure 6: Graph of various Sorting Algorithms with Random order of elements as input

Correlation between Number of Comparisions with Execution Time

Figure 7 illustrates the relationship between the number of comparisons and execution time for various sorting algorithms.

From the graph, a strong positive linear correlation is clearly observed for all comparison-based sorting algorithms. The correlation coefficients for Selection Sort, Bubble Sort, and Insertion Sort are extremely close to 1 (approx. 1.000), indicating an almost perfect linear relationship between the number of comparisons and execution time. This confirms that for these algorithms, execution time is heavily dominated by the number of comparisons performed.

Similarly, efficient algorithms such as Merge Sort, Quick Sort (median pivot), and Heap Sort also exhibit a strong correlation (approx. 0.996–0.999). Although slightly lower than the quadratic algorithms, these values still indicate that the number of comparisons is a major factor influencing execution time.

The linear trend observed in all plots suggests that as the number of comparisons increases, execution time increases proportionally. This validates the theoretical understanding that comparisons are a primary contributor to the time complexity of comparison-based sorting algorithms.

However, it is also evident that different algorithms have different slopes in the graph, indicating varying constant factors and overheads. For example, even with a similar number of comparisons, Merge Sort and Heap Sort may take slightly different amounts of time due to differences in memory access patterns and implementation details.

Overall, Figure 7 confirms that while execution time is strongly correlated with the number of comparisons, it is not solely determined by it. Other factors such as recursion overhead, cache efficiency, and data movement also contribute to the observed performance. Nonetheless, the number of comparisons remains a reliable indicator of algorithm efficiency and aligns closely with the theoretical time complexities.

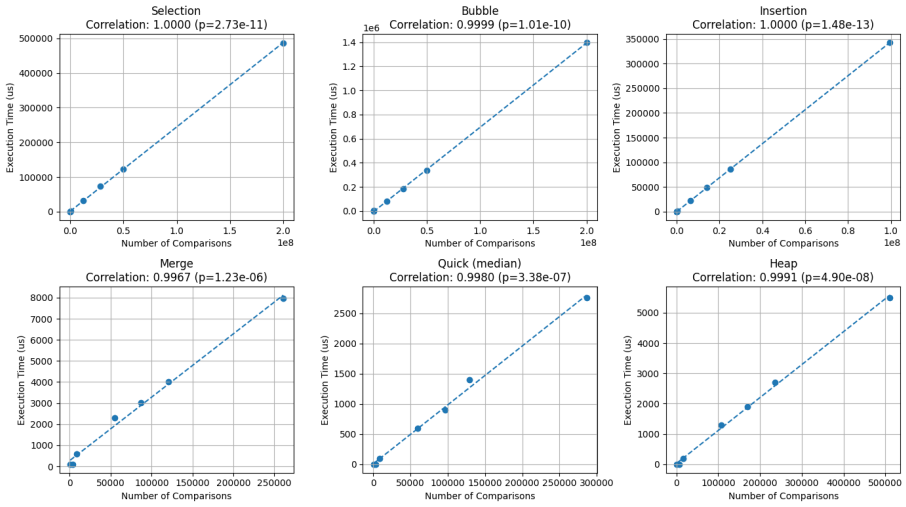

Figure 7: Graph of Correlation between Number of Comparisions with Execution Time